

ARTI-AIRLINK — GUIDE TECHNIQUE COMPLET

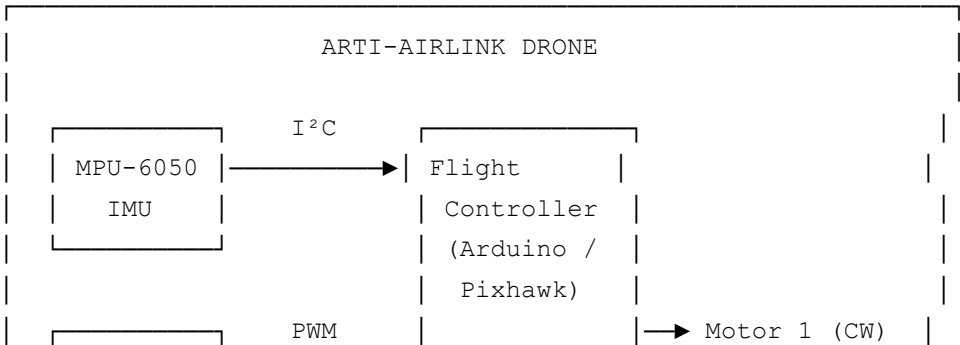
IMU · Contrôleur PID · Système de Largage Servo

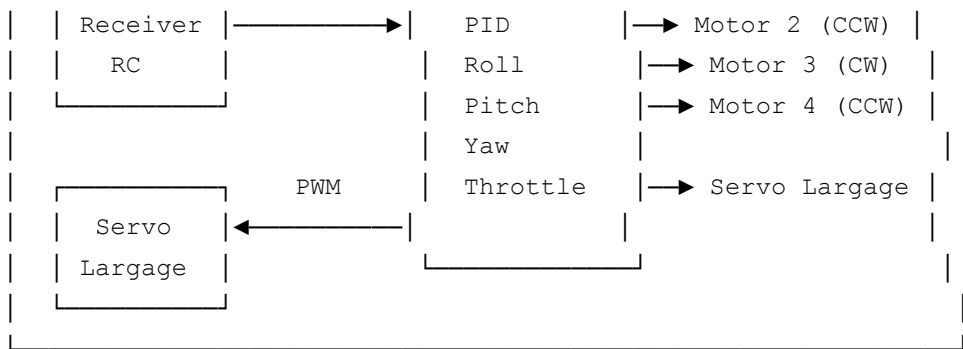
Version 2.0 — Équipe Arti | Section Drone & Innovation

TABLE DES MATIÈRES

- 1. Architecture technique globale
- 2. Diagnostic du drone jouet initial
- 3. Intégration IMU (MPU-6050 / ICM-42688-P)
- 4. Contrôleur PID — Théorie et implémentation
- 5. Code Arduino complet — Stabilisation
- 6. Code Python — Simulation et calibration PID
- 7. Système de largage servo (Payload Release)
- 8. Schémas de câblage
- 9. Séquence de tests progressive
- 10. Annexe — Liens et ressources

1. ARCHITECTURE TECHNIQUE GLOBALE





Flux de données :

1. IMU mesure orientation (roulis, tangage, lacet) à 1000 Hz
2. Filtre complémentaire fusionne gyro + accéléro
3. PID calcule correction pour chaque axe
4. ESC ajuste vitesse des 4 moteurs
5. Servo se déclenche sur commande RC ou mission

2. DIAGNOSTIC DU DRONE JOUET INITIAL

2.1 Problèmes identifiés

Problème	Impact	Solution
Absence de gyroscope	Instabilité totale en vol	Ajouter IMU externe
Centre de gravité décalé	Dérive constante	Recentrer batterie
Moteurs asymétriques	Rotation non désirée	Calibrer / remplacer
Hélices déséquilibrées	Vibrations excessives	Équilibrer avec ruban
Câblage non fixé	Vibrations parasites	Fixer avec colliers
Soudures douteuses	Pertes de puissance	Reprendre les soudures

2.2 Check-list mécanique avant tout vol

- [] Centre de gravité vérifié (banc d'équilibre)
- [] Hélices CW/CCW dans le bon sens

- [] Chaque hélice tournée à la main – pas de frottement
- [] Câbles fixés – aucun fil libre
- [] Batterie centrée et fixée
- [] Soudures inspectées visuellement
- [] Test à vide moteur par moteur (ESC calibré)

3. INTÉGRATION IMU — MPU-6050

3.1 Comparatif des IMU recommandées

Capteur	Interface	Axes	Bruit	Prix	Recommandé pour
MPU-6050 (GY-521)	I ² C	6 (3G+3A)	Moyen	~3€	Début, prototypage
ICM-42688-P	SPI/I ² C	6	Très faible	~8€	Production
MPU-6886	I ² C	6	Faible	~5€	Intermédiaire

Choix recommandé Phase 1 : MPU-6050 (GY-521) Choix recommandé Phase 2 : ICM-42688-P

3.2 Schéma de câblage MPU-6050 → Arduino

MPU-6050 (GY-521)	Arduino Uno / Nano
VCC	3.3V (JAMAIS 5V!)
GND	GND
SCL	A5 (SCL)
SDA	A4 (SDA)
INT	D2 (interruption externe)
AD0	GND (adresse I ² C = 0x68)

Résistances pull-up recommandées :

SCL —[4.7kΩ]— 3.3V

SDA —[4.7kΩ]— 3.3V

Anti-vibration :

- └─ Monter le module sur mousse fine (3M VHB ou équivalent)
épaisseur 2-3mm, pas plus (trop mou = perte de signal)

3.3 Registres clés MPU-6050

Registre 0x6B : PWR_MGMT_1 → Réveil du capteur
Registre 0x1B : GYRO_CONFIG → Plage gyro (± 250 , ± 500 , ± 1000 , ± 2000 °/s)
Registre 0x1C : ACCEL_CONFIG → Plage accéléro ($\pm 2g$, $\pm 4g$, $\pm 8g$, $\pm 16g$)
Registres 0x3B-0x48 : données brutes (6 axes $\times$ 2 octets)

4. CONTRÔLEUR PID — THÉORIE

4.1 Principe général

Le PID calcule une correction $u(t)$ à partir de l'erreur $e(t) = \text{consigne} - \text{mesure}$:

$$u(t) = \underbrace{K_p \cdot e(t)}_{\text{Proportionnel}} + \underbrace{K_i \cdot \int e(t) dt}_{\text{Intégral}} + \underbrace{K_d \cdot de(t)/dt}_{\text{Dérivé}}$$

K_p : Réaction immédiate proportionnelle à l'erreur
 K_i : Correction des erreurs accumulées (dérive lente)
 K_d : Amortissement — anticipe les changements rapides

4.2 Effet de chaque terme sur le drone

Terme	Trop faible	Trop élevé
K_p	Réaction lente, grosse dérive	Oscillations rapides
K_i	Drift non corrigé	Windup — instabilité lente
K_d	Pas d'amortissement	Amplification du bruit

4.3 Valeurs de départ recommandées (quadricoptère 250mm)

Axe Roll : $K_p=1.5$ $K_i=0.05$ $K_d=12$
Axe Pitch : $K_p=1.5$ $K_i=0.05$ $K_d=12$
Axe Yaw : $K_p=2.5$ $K_i=0.05$ $K_d=0$

Procédure de tuning :

- Commencer avec $K_i=0$ et $K_d=0$
- Augmenter K_p jusqu'au début des oscillations, puis réduire de 30%

3. Augmenter Kd pour amortir les oscillations
4. Ajouter Ki doucement pour éliminer le drift

5. CODE ARDUINO — STABILISATION COMPLÈTE

```
/*
 * ARTI-AIRLINK — Contrôleur de vol minimal
 * IMU : MPU-6050 via I²C
 * Contrôle : PID sur Roll, Pitch, Yaw
 * Auteur : Équipe Arti | Section Drone & Innovation
 * Version : 2.0
 */

#include <Wire.h>

// — Constantes IMU —————
#define MPU_ADDR      0x68
#define GYRO_SCALE     65.5    // LSB/(°/s) pour ±500°/s
#define ACCEL_SCALE    8192.0  // LSB/g pour ±4g
#define RAD_TO_DEG     57.2957795

// — Broches ESC (PWM) —————
#define ESC_FL  9    // Avant-Gauche  (CW)
#define ESC_FR  10   // Avant-Droit   (CCW)
#define ESC_BL  11   // Arrière-Gauche (CCW)
#define ESC_BR  6    // Arrière-Droit  (CW)

// — Broche servo largage —————
#define SERVO_PIN  5
#define SERVO_HOLD 90 // Angle de maintien (colis retenu)
#define SERVO_DROP 0  // Angle de largage

// — Paramètres PID —————
struct PID_Params {
    float kp, ki, kd;
    float integral;
    float prev_error;
    float output;
};

PID_Params pid_roll  = {1.5, 0.05, 12.0, 0, 0, 0};
PID_Params pid_pitch = {1.5, 0.05, 12.0, 0, 0, 0};
PID_Params pid_yaw   = {2.5, 0.05, 0.0, 0, 0, 0};
```

```

// — Variables globales —————
float accel_x, accel_y, accel_z;
float gyro_x, gyro_y, gyro_z;
float angle_roll = 0, angle_pitch = 0, angle_yaw = 0;
float gyro_offset_x = 0, gyro_offset_y = 0, gyro_offset_z = 0;

float setpoint_roll = 0; // Consigne (depuis RC)
float setpoint_pitch = 0;
float setpoint_yaw = 0;
int throttle = 1000; // µs PWM (1000 = arrêté, 2000 = max)

unsigned long last_time = 0;
float dt = 0;

// — Initialisation —————
void setup() {
  Serial.begin(115200);
  Wire.begin();
  Wire.setClock(400000); // I²C fast mode 400kHz

  // Réveil du MPU-6050
  Wire.beginTransmission(MPU_ADDR);
  Wire.write(0x6B);
  Wire.write(0x01); // Clock PLL gyro X
  Wire.endTransmission();

  // Plage gyro ±500°/s
  Wire.beginTransmission(MPU_ADDR);
  Wire.write(0x1B);
  Wire.write(0x08);
  Wire.endTransmission();

  // Plage accéléro ±4g
  Wire.beginTransmission(MPU_ADDR);
  Wire.write(0x1C);
  Wire.write(0x08);
  Wire.endTransmission();

  // Filtre passe-bas DLPF à 42Hz
  Wire.beginTransmission(MPU_ADDR);
  Wire.write(0x1A);
  Wire.write(0x03);
  Wire.endTransmission();

  delay(100);
  calibrate_gyro();
}

```

```

// Servo initial - position maintien
servo_set(SERVO_HOLD);

// ESC - signal minimal
analogWrite(ESC_FL, 0);
analogWrite(ESC_FR, 0);
analogWrite(ESC_BL, 0);
analogWrite(ESC_BR, 0);

last_time = micros();
Serial.println("ARTI-AIRLINK READY");
}

// — Boucle principale —————
void loop() {
    // Calcul dt
    unsigned long now = micros();
    dt = (now - last_time) / 1000000.0;
    last_time = now;

    // 1. Lecture IMU
    read_imu();

    // 2. Fusion d'attitude (filtre complémentaire)
    float accel_angle_roll = atan2(accel_y, accel_z) * RAD_TO_DEG;
    float accel_angle_pitch = atan2(-accel_x, sqrt(accel_y*accel_y + accel_z*accel_z));

    // Filtre complémentaire : 98% gyro + 2% accéléro
    angle_roll = 0.98 * (angle_roll + gyro_x * dt) + 0.02 * accel_angle_roll;
    angle_pitch = 0.98 * (angle_pitch + gyro_y * dt) + 0.02 * accel_angle_pitch;
    angle_yaw += gyro_z * dt;

    // 3. Calcul PID
    compute_pid(pid_roll, setpoint_roll - angle_roll, dt);
    compute_pid(pid_pitch, setpoint_pitch - angle_pitch, dt);
    compute_pid(pid_yaw, setpoint_yaw - angle_yaw, dt);

    // 4. Mixage moteurs (quadricoptère configuration X)
    int fl = throttle - pid_roll.output + pid_pitch.output - pid_yaw.output;
    int fr = throttle + pid_roll.output + pid_pitch.output + pid_yaw.output;
    int bl = throttle - pid_roll.output - pid_pitch.output + pid_yaw.output;
    int br = throttle + pid_roll.output - pid_pitch.output - pid_yaw.output;

    // 5. Sortie ESC (contraindre entre 1000 et 2000µs)
    set_esc(ESC_FL, constrain(fl, 1000, 2000));
    set_esc(ESC_FR, constrain(fr, 1000, 2000));
    set_esc(ESC_BL, constrain(bl, 1000, 2000));

```

```

set_esc(ESC_BR, constrain(br, 1000, 2000));

// 6. Debug Serial (désactiver en production)
if (millis() % 100 == 0) {
    Serial.print("R:"); Serial.print(angle_roll, 1);
    Serial.print(" P:"); Serial.print(angle_pitch, 1);
    Serial.print(" Y:"); Serial.println(angle_yaw, 1);
}
}

// — Lecture IMU —————
void read_imu() {
    Wire.beginTransaction(MPU_ADDR);
    Wire.write(0x3B); // Registre ACCEL_XOUT_H
    Wire.endTransmission(false);
    Wire.requestFrom(MPU_ADDR, 14, true);

    accel_x = (Wire.read() << 8 | Wire.read()) / ACCEL_SCALE;
    accel_y = (Wire.read() << 8 | Wire.read()) / ACCEL_SCALE;
    accel_z = (Wire.read() << 8 | Wire.read()) / ACCEL_SCALE;
    Wire.read(); Wire.read(); // température (skip)
    gyro_x = (Wire.read() << 8 | Wire.read()) / GYRO_SCALE - gyro_offset_x;
    gyro_y = (Wire.read() << 8 | Wire.read()) / GYRO_SCALE - gyro_offset_y;
    gyro_z = (Wire.read() << 8 | Wire.read()) / GYRO_SCALE - gyro_offset_z;
}

// — Calibration gyroscope (500 échantillons) —————
void calibrate_gyro() {
    Serial.println("Calibration gyro - Ne pas bouger le drone...");
    long sum_x = 0, sum_y = 0, sum_z = 0;
    for (int i = 0; i < 500; i++) {
        Wire.beginTransaction(MPU_ADDR);
        Wire.write(0x43);
        Wire.endTransmission(false);
        Wire.requestFrom(MPU_ADDR, 6, true);
        sum_x += (Wire.read() << 8 | Wire.read());
        sum_y += (Wire.read() << 8 | Wire.read());
        sum_z += (Wire.read() << 8 | Wire.read());
        delay(3);
    }
    gyro_offset_x = sum_x / 500.0 / GYRO_SCALE;
    gyro_offset_y = sum_y / 500.0 / GYRO_SCALE;
    gyro_offset_z = sum_z / 500.0 / GYRO_SCALE;
    Serial.println("Calibration terminée ✓");
}

// — Calcul PID —————

```



```

void compute_pid(PID_Params &p, float error, float dt) {
    // Anti-windup : limiter l'intégrale
    p.integral += error * dt;
    p.integral = constrain(p.integral, -400, 400);

    float derivative = (error - p.prev_error) / dt;
    p.output = p.kp * error + p.ki * p.integral + p.kd * derivative;
    p.output = constrain(p.output, -400, 400);
    p.prev_error = error;
}

// — Contrôle ESC (1000-2000µs en PWM simulé) —————
void set_esc(int pin, int value_us) {
    // Conversion µs → duty cycle 8 bits pour analogWrite
    // 1000µs = 51, 2000µs = 102 à 490Hz
    int duty = map(value_us, 1000, 2000, 51, 102);
    analogWrite(pin, duty);
}

// — Contrôle servo largage —————
void servo_set(int angle) {
    // Génération du signal servo sans bibliothèque Servo
    int pulse_us = map(angle, 0, 180, 1000, 2000);
    int duty = map(pulse_us, 1000, 2000, 51, 102);
    analogWrite(SERVO_PIN, duty);
}

// — Déclencher le largage —————
void trigger_drop() {
    servo_set(SERVO_DROP);
    delay(500);
    servo_set(SERVO_HOLD); // Retour position initiale
    Serial.println("LARGAGE EFFECTUÉ");
}

```

6. CODE PYTHON — SIMULATION ET TUNING PID

```

"""
ARTI-AIRLINK — Simulateur PID
Permet de tuner les paramètres Kp, Ki, Kd avant de les flasher sur Arduino
Auteur : Équipe Arti | Section Drone & Innovation
"""

import numpy as np

```

```

import matplotlib.pyplot as plt
from dataclasses import dataclass, field

@dataclass
class PIDController:
    kp: float = 1.5
    ki: float = 0.05
    kd: float = 12.0
    integral: float = 0.0
    prev_error: float = 0.0
    integral_limit: float = 400.0

    def compute(self, setpoint: float, measured: float, dt: float) -> float:
        error = setpoint - measured
        self.integral += error * dt
        self.integral = np.clip(self.integral, -self.integral_limit, self.integral_limit)
        derivative = (error - self.prev_error) / dt if dt > 0 else 0
        output = self.kp * error + self.ki * self.integral + self.kd * derivative
        self.prev_error = error
        return np.clip(output, -400, 400)

@dataclass
class DroneAxis:
    """Modèle simplifié d'un axe de rotation du drone"""
    inertia: float = 0.01          # Moment d'inertie (kg·m²)
    damping: float = 0.005         # Frottement visqueux
    angle: float = 0.0
    angular_velocity: float = 0.0

    def update(self, torque: float, dt: float):
        angular_accel = (torque - self.damping * self.angular_velocity) / self.inertia
        self.angular_velocity += angular_accel * dt
        self.angle += self.angular_velocity * dt
        return self.angle

def simulate_pid(kp=1.5, ki=0.05, kd=12.0, duration=5.0, dt=0.001,
                setpoint=0.0, initial_disturbance=10.0):
    """
    Simule la réponse PID d'un axe drone

    Paramètres:
        kp, ki, kd      : Gains PID
        duration         : Durée simulation (s)
        dt               : Pas de temps (s)
        setpoint         : Angle cible (°)
    """

```

```

        initial_disturbance : Perturbation initiale (°)
    """
    pid = PIDController(kp=kp, ki=ki, kd=kd)
    axis = DroneAxis(angle=initial_disturbance)

    times, angles, errors, outputs = [], [], [], []
    t = 0.0

    while t < duration:
        current_angle = axis.angle
        control = pid.compute(setpoint, current_angle, dt)

        # Convertir la sortie PID en couple moteur (scaling)
        torque = control * 0.001
        axis.update(torque, dt)

        times.append(t)
        angles.append(current_angle)
        errors.append(setpoint - current_angle)
        outputs.append(control)
        t += dt

    return np.array(times), np.array(angles), np.array(errors), np.array(outputs)

def plot_comparison():
    """Compare différentes configurations PID"""
    configs = [
        {"kp": 0.5, "ki": 0.01, "kd": 2.0, "label": "Kp faible (sous-amorti)",
         "color": "blue", "dash": [5, 5]},
        {"kp": 1.5, "ki": 0.05, "kd": 12.0, "label": "Config recommandée",
         "color": "red", "dash": [1, 0]},
        {"kp": 5.0, "ki": 0.1, "kd": 30.0, "label": "Kp élevé (sur-amorti)",
         "color": "green", "dash": [5, 5]},
        {"kp": 1.5, "ki": 2.0, "kd": 12.0, "label": "Ki élevé (windup intégral)",
         "color": "black", "dash": [1, 0]}
    ]

    fig, axes = plt.subplots(2, 1, figsize=(12, 8))
    fig.suptitle("ARTI-AIRLINK – Comparaison configurations PID\n(Perturbation in
                 fontsize=13, fontweight='bold')

    for cfg in configs:
        t, angles, errors, outputs = simulate_pid(
            kp=cfg["kp"], ki=cfg["ki"], kd=cfg["kd"]
        )
        axes[0].plot(t, angles, label=cfg["label"], color=cfg["color"], linewidth=1)
        axes[1].plot(t, errors, color=cfg["color"], linewidth=1.5, alpha=0.7)

    axes[0].axhline(y=0, color='black', linestyle='--', linewidth=1, alpha=0.5)
    axes[0].set_ylabel("Angle (°)", fontsize=11)

```

```

axes[0].set_title("Réponse angulaire", fontsize=11)
axes[0].legend(loc='upper right', fontsize=9)
axes[0].grid(True, alpha=0.3)
axes[0].set_ylim(-3, 12)

axes[1].axhline(y=0, color='black', linestyle='--', linewidth=1, alpha=0.5)
axes[1].set_xlabel("Temps (s)", fontsize=11)
axes[1].set_ylabel("Erreur (°)", fontsize=11)
axes[1].set_title("Erreur résiduelle", fontsize=11)
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig("arti_airlink_pid_comparison.png", dpi=150, bbox_inches='tight')
plt.show()
print("Graphique sauvegardé : arti_airlink_pid_comparison.png")

def tune_interactive():
    """Tuning interactif - tester vos valeurs"""
    print("=" * 50)
    print("ARTI-AIRLINK - Tuning PID Interactif")
    print("=" * 50)
    print("Entrez vos valeurs (Entrée = valeur par défaut)")

    kp = float(input("Kp [1.5] : ") or 1.5)
    ki = float(input("Ki [0.05]: ") or 0.05)
    kd = float(input("Kd [12.0]: ") or 12.0)

    t, angles, errors, _ = simulate_pid(kp=kp, ki=ki, kd=kd)

    # Métriques de performance
    overshoot = (max(abs(angles)) - 0) / 10.0 * 100 # % du déplacement initial
    settling_idx = np.where(np.abs(errors) < 0.5)[0]
    settling_time = t[settling_idx[0]] if len(settling_idx) > 0 else float('inf')
    steady_error = abs(np.mean(errors[-100:])) # Erreur moyenne sur les 100 dern

    print(f"\n--- Résultats ---")
    print(f"Dépassement max : {overshoot:.1f}%")
    print(f"Temps de stabilisation (±0.5°) : {settling_time:.3f}s")
    print(f"Erreur statique : {steady_error:.4f}°")

    if overshoot > 20:
        print("⚠ Dépassement trop élevé - augmenter Kd ou réduire Kp")
    if settling_time > 2.0:
        print("⚠ Trop lent - augmenter Kp")
    if steady_error > 0.1:
        print("⚠ Erreur résiduelle - augmenter Ki")

```

```

    if overshoot < 5 and settling_time < 0.5:
        print("✅ Configuration optimale !")

    return kp, ki, kd

if __name__ == "__main__":
    print("ARTI-AIRLINK – Simulateur PID")
    print("1. Afficher comparaison configurations")
    print("2. Tuning interactif")
    choix = input("Choix [1/2] : ")

    if choix == "1":
        plot_comparison()
    elif choix == "2":
        kp, ki, kd = tune_interactive()
        print(f"\nValeurs à flasher sur Arduino :")
        print(f"PID_Params pid_roll = {{{kp}}, {{ki}}, {{kd}}, 0, 0, 0}};")
        print(f"PID_Params pid_pitch = {{{kp}}, {{ki}}, {{kd}}, 0, 0, 0}};")

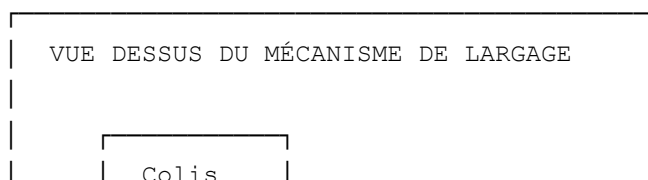
```

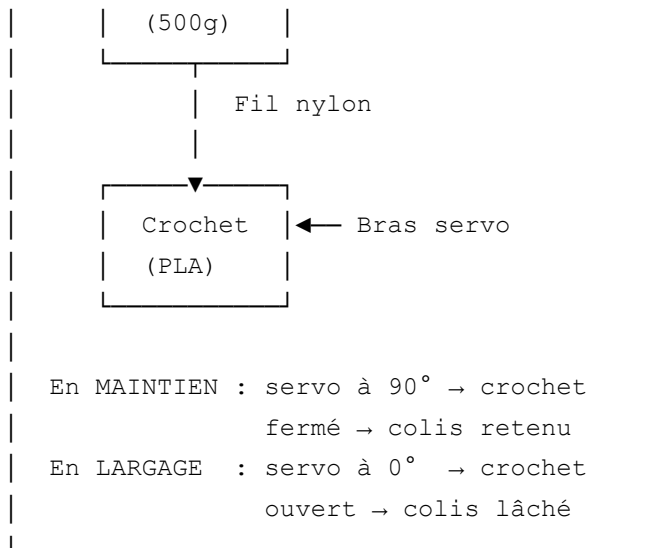
7. SYSTÈME DE LARGAGE SERVO — PAYLOAD RELEASE

7.1 Composants nécessaires

Composant	Référence	Prix approx.	Fournisseur
Micro-servo	SG90 ou MG90S (métal)	2–4€	Conrad, AliExpress
Crochet	Imprimé 3D PLA	< 1€	Impression locale
Fil Nylon	0.5mm, résistance 5kg	2€	Quincaillerie
UBEC 5V	3A mini	4€	AliExpress / TME
Connecteur XT30	Pour charge	1€	Conrad

7.2 Mécanisme de largage





7.3 Fichier STL — Description du crochet à imprimer

Dimensions recommandées :

- Longueur totale : 35mm
- Diamètre axe servo : 5mm (adapter selon modèle)
- Largeur rainure charge : 4mm
- Matériau : PLA+ ou PETG
- Remplissage : 40% minimum
- Périmètres : 3 minimum

Logiciels de modélisation gratuits :

- Tinkercad (débutant, en ligne)
- FreeCAD (avancé, gratuit)
- Fusion 360 (pro, gratuit étudiant)

8. SÉQUENCE DE TESTS PROGRESSIVE

Phase A — Banc sans hélices

ÉTAPE 1 : Test IMU seul

- ☐ Connecter MPU-6050 à Arduino
- ☐ Ouvrir Serial Monitor (115200 baud)
- ☐ Vérifier lecture Roll/Pitch/Yaw en temps réel
- ☐ Incliner le drone manuellement
- ☐ Validation : angles cohérents et stables

ÉTAPE 2 : Test servo largage

- ☐ Brancher servo sur pin 5
- ☐ Tester SERVO_HOLD (90°) → crochet fermé
- ☐ Tester SERVO_DROP (0°) → crochet ouvert
- ☐ Fixer un colis de 300g, déclencher largage
- ☐ Validation : largage propre, retour position

ÉTAPE 3 : Test PID sans hélices

- ☐ Flasher le code complet
- ☐ Observer les corrections PID dans Serial Monitor
- ☐ Incliner le drone → vérifier corrections opposées
- ☐ Validation : PID réagit dans le bon sens

Phase B — Vol stationnaire bas (50cm)

ÉTAPE 4 : Premier vol sans charge

- ☐ Zone dégagée, 3m x 3m minimum
- ☐ Monter progressivement les gaz (throttle)
- ☐ Observer la stabilité
- ☐ Ajuster Kp si oscillations

ÉTAPE 5 : Vol avec charge 200g

- ☐ Fixer charge légère (200g) sur crochet servo
- ☐ Répéter vol stationnaire
- ☐ Observer l'impact de la charge sur PID
- ☐ Ajuster si nécessaire

ÉTAPE 6 : Test largage en vol bas

- ☐ Vol à 1m de hauteur
- ☐ Déclencher largage via commande RC
- ☐ Observer stabilité après largage (centre de gravité change !)
- ☐ Ajuster PID pour compensation post-largage

Phase C — Validation opérationnelle

ÉTAPE 7 : Vol avec charge cible 500g

ÉTAPE 8 : Test endurance (autonomie batterie sous charge)

ÉTAPE 9 : Test largage précision (cible au sol)

ÉTAPE 10 : Documentation résultats → rapport technique Arti-AirLink

9. RESSOURCES ET LIENS

Composants (Belgique / Europe)

Fournisseur	URL	Spécialité
Conrad	conrad.be	Électronique générale
TME	tme.eu	Composants pro, Arduino
Gotronic	gotronic.fr	Robots, capteurs
AliExpress	aliexpress.com	Prix bas, délai 2–4 sem.
Tinytronics	tinytronics.nl	Arduino/Raspberry, Pays-Bas

Bibliothèques Arduino utiles

```
// Dans l'IDE Arduino – Gestionnaire de bibliothèques :  
MPU6050          // par Electronic Cats  
Wire             // inclus par défaut  
Servo            // inclus par défaut (alternative à analogWrite)
```

Outils de simulation

```
# Installation Python  
pip install numpy matplotlib scipy  
  
# Simulateur PID (code fourni ci-dessus)  
python arti_airlink_pid_sim.py
```

Réglementation (Belgique)

Organisme	Contact	URL
DGTA	dgta@mobilit.fgov.be	mobilit.fgov.be
Formation A1/A3 gratuite	Via DGTA	belgianuas.be
Assurance RC drone	AG Insurance, AXA	—

